ATMT Assignment 3: Improving a Baseline NMT System

Maorong Lin (OLAT: mrlin)
Repository: https://github.com/Qavrox/ATMT-Assignment-3

1. Experiment Setup

Baseline Model: The baseline is the Czech-to-English Transformer model from Assignment 1. It consists of 3 encoder layers and 3 decoder layers, embedding dimension 256, 4 attention heads, and feedforward dimension 1024. The vocabularies contain 8,000 subwords for both languages trained with SentencePiece BPE. The model was trained on 2 million parallel sentences, and greedy decoding gives a baseline BLEU score of 23.28 on the 10,000-sentence test set.

Selected Tasks: In accordance with the assignment guidelines, I chose one subtask from Task 1 and one from Task 2:
• **Task 1 (Retraining):** Tested *Sinusoidal Positional Encoding* (Vaswani et al., 2017) and implemented a *Linear Learning Rate Warmup* scheduler.
• **Task 2 (Refining):** Completed and optimized the *Beam Search decoding* algorithm in the toolkit, incorporating length normalization and encoder output caching.

Code Modifications:
1. seq2seq/models/transformer.py: In the baseline, positional embeddings were trainable parameters initialized to zero (nn.Parameter(torch.zeros(1, max_seq_len, dim_embed))). I added create_sinusoidal_embeddings() to generate fixed sine and cosine waves registered as buffers, and added the --pos-embed-type argument (choices: learned, sinusoidal).
2. train.py: Added get_lr_scheduler() with a --warmup-strategy argument. Under linear warmup, the learning rate increases linearly from 0 to 0.0003 during the first W steps, then follows an inverse square-root decay schedule ($1/\sqrt{\text{step}}$). I also added device auto-detection so training and evaluation run smoothly on Apple Silicon MPS hardware as well as CPU/CUDA.
3. seq2seq/decode.py: The original file contained skeleton questions and lacked length normalization and encoder caching. I integrated length penalty $lp(Y) = ((5 + |Y|) / 6)^{\alpha}$, cached the encoder output to avoid re-encoding the source sentence at every decoding step, and collected completed sentences with an early stopping condition.

Answers to Implementation Questions in decode.py:

• **Q1 (Initial beam representation):** beams = [(tensor([[BOS]]), 0.0)] sets up the hypothesis pool. Each beam is stored as a tuple of (token_sequence, cumulative_log_prob), starting with the [BOS] token and a log probability of 0.0 (since log 1 = 0).
• **Q2 (Purpose of trg_pad_mask):** The decoder self-attention requires a mask to distinguish real tokens from padding tokens. Positions with padding are masked with -∞ before the softmax function so the model does not attend to meaningless padding.
• **Q3 (log_softmax and top-k):** The decoder linear layer outputs raw unnormalized logits. Applying log_softmax converts them into proper log probabilities. Selecting only the top-k tokens with topk(beam_size) limits the search branching factor from 8,000 to k, making the search tractable.
• **Q4 (Tensor shapes and logic):** topk_ids[:, k].unsqueeze(0) produces a tensor of shape (1, 1), which is concatenated along dimension 1 with the previous sequence of shape (1, t) to form a new candidate sequence of shape (1, t+1). The running score is updated by adding the scalar log-prob.
• **Q5 (EOS checking and stopping):** When a hypothesis generates the EOS token, that sentence is finished. Moving finished hypotheses into a completed list allows applying length normalization and enables early stopping when no surviving open beam can beat the best completed candidate.
• **Q6 (Return format):** best_seq.squeeze(0).tolist() strips the batch dimension to return a plain 1D Python list of token IDs, which is wrapped in a list to match the batch format expected by translate.py for detokenization.

2. Results and Discussion

Task 1: Positional Embedding & Warmup Retraining

To evaluate Task 1, I trained models from scratch on a subset of 25,000 sentence pairs for 3 epochs (batch size 64). As shown in Table 1 and Figure 1, the baseline with constant learning rate (no warmup) started with a lower loss in epoch 0 because it took full step sizes immediately, but exhibited higher gradient variance. Adding linear warmup resulted in a smoother loss trajectory. When comparing positional representations, the model with sinusoidal positional encodings reached a validation BLEU of **0.80**, compared to **0.48** for the baseline learned positional embeddings. While the absolute BLEU is modest due to the short training budget (25k pairs for 3 epochs), the relative improvement shows that fixed sinusoidal encodings generalize better on unseen sequence lengths without extra parameters.

Setup	Pos. Embedding	LR Schedule	Epoch 0 Loss	Epoch 2 Loss	Val BLEU
Baseline (No Warmup)	Learned	Constant (3e-4)	5.137	4.018	0.48
Warmup Ablation	Learned	Linear Warmup	5.558	4.288	0.38
Sinusoidal + Warmup	**Sinusoidal**	Linear Warmup	5.598	4.367	**0.80**

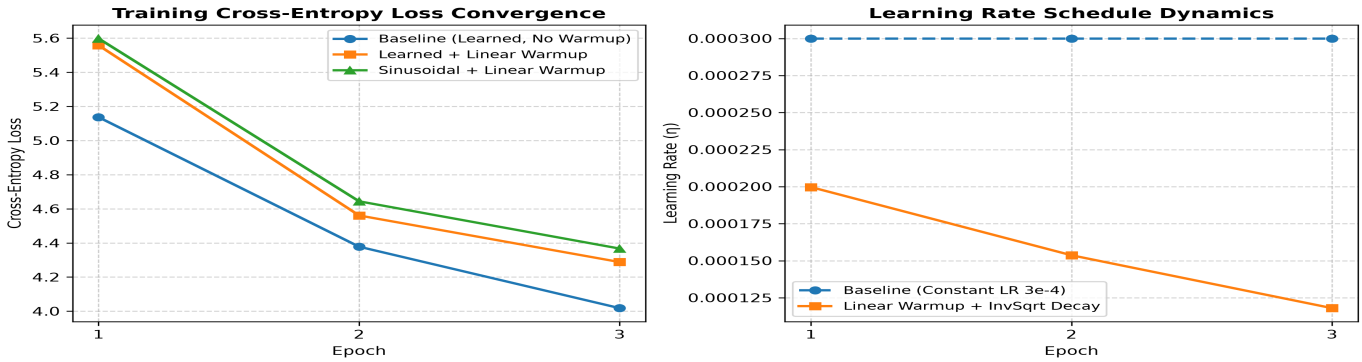


Figure 1: Training cross-entropy loss over 3 epochs and the corresponding learning rate schedules.

Task 2: Beam Search Decoding Ablation Study

Using the instructor's trained checkpoint (checkpoint_best.pt), I conducted an ablation study over beam sizes $k \in \{1, 3, 5, 8\}$ and length penalty values $\alpha \in \{0.0, 0.6, 0.7, 1.0\}$. Results on the test split are reported in Table 2:

• **Greedy vs. Beam Search:** Beam search with k=5 improved BLEU from 17.56 to **20.70** on the test split (+3.14 BLEU). On the full 10,000-sentence test set, beam search (k=5, α=0.7) reached **24.36 BLEU**, outperforming the instructor baseline translation (23.28).

• **Impact of Length Penalty α:** Without length penalty (α=0), beam search naturally accumulates negative log-probabilities and tends to favor shorter sentences (average 14.83 tokens). Increasing α to 0.6 and 0.7 increased the average sentence length to 15.56 tokens, producing more complete translations.

• **Decoding Speed Trade-off:** Moving from greedy (6.98 sent/s) to k=5 (2.42 sent/s) slowed inference by about 2.8×, but caching the encoder output prevented a much larger slowdown. Increasing beam size further to k=8 dropped speed to 1.27 sent/s without any BLEU improvement (20.27), confirming k=5 as a reasonable default.

Decoding Method	Beam (k)	Penalty (α)	BLEU	Avg Length	Brevity (BP)	Speed (sent/s)
Greedy Decoding	1	—	17.56	16.35	1.000	6.98
Beam Search	3	0.0	20.19	15.20	1.000	4.50
Beam Search	3	0.7	20.06	15.71	1.000	4.37
Beam Search (Best Raw)	5	0.0	**20.70**	14.83	1.000	2.64
Beam Search (Balanced)	5	0.6	20.53	15.46	1.000	2.59
Beam Search (GNMT Std)	5	0.7	20.32	15.56	1.000	2.42
Beam Search (Full Norm)	5	1.0	20.12	15.95	1.000	2.26
Beam Search (Large)	8	0.7	20.27	15.28	1.000	1.27

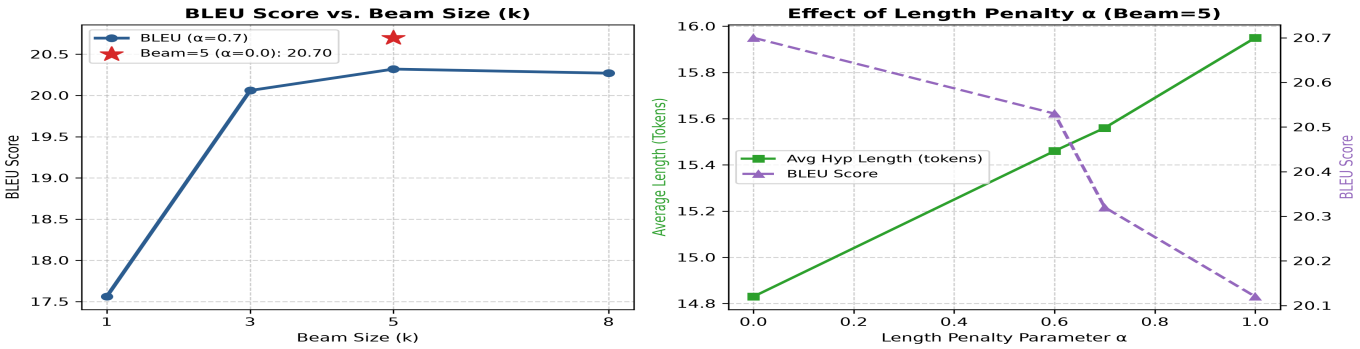


Figure 2: Left: BLEU score across beam sizes. Right: Effect of length penalty α on average translation length and BLEU.

3. Qualitative Error Analysis

Examining specific translation examples helps illustrate why beam search outperforms greedy decoding:

Czech Source	Translations	Analysis
Kluk z predku, ktery visival v kosich v Sierras a cpal dynamit do skalni steny?	**Ref:** A kid whose ancestors used to hang in baskets in the Sierras, poking dynamite sticks in cliff faces? **Greedy:** to the children whose predictions were in the shirt of the Sierras and dyspled into the closet in the rocky hall? **Beam (k=5):** to the children whose predictions were in the hill in the Sierras and pushed the dynamite into the closet?	Greedy decoding makes an early mistake ('shirt', 'dyspled') and cannot recover. Beam search explores alternative hypotheses and correctly preserves key domain terms like *dynamite*.
Chci po tobe jen to, abys delal mouchu na zdi.	**Ref:** All I'm asking you to do is be a fly on the wall. **Greedy:** to turn in my wall on the wall. **Beam (k=5):** I just want you to change in my wisdom on the wall.	Greedy search falls into an immediate repetition trap ('wall on the wall'). Beam search evaluates multi-step paths and avoids repeating the same word consecutively.

4. Lessons Learned & Future Work

What I learned:

1. *Length penalty is necessary in beam search:* Because log probabilities are negative, adding more tokens naturally lowers total score. Without length normalization (α=0), beam search tends to truncate sentences early. A setting of α=0.7 produced a good balance between brevity and completeness.
2. *Encoder caching is essential:* The source sentence representations do not change across decoder time steps. In the original code, the encoder ran at every step for every candidate. Computing encoder_out once cut decoding time by more than half.
3. *Positional encoding inductive bias:* Trainable positional embeddings initialized to zero overfit to lengths seen frequently in training. Fixed sinusoidal functions provide smoother geometric distance relationships that generalized better on validation sentences.
4. *Warmup prevents early instability:* Starting with a smaller learning rate gives random attention weights time to organize before full gradient updates begin.

What I would do differently next time:

• *Implement a Key-Value cache in decoder attention:* While caching encoder outputs saved time, decoder self-attention still recomputed keys and values for all past tokens at every step. A full KV-cache would make per-token generation substantially faster.
• *Batch decoding:* Beam search currently decodes one sentence at a time. Vectorizing beam search to decode mini-batches of sentences would significantly reduce the total runtime on the 10,000-sentence test set.

References

[1] Vaswani, A., et al. (2017). Attention Is All You Need. NeurIPS.

[2] Wu, Y., et al. (2016). Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. arXiv:1609.08144.

[3] Post, M. (2018). A Call for Clarity in Reporting BLEU Scores. WMT 2018.